

Practical: 1

Aim: Using Python, demonstrate the functionality of NumPy, Pandas, and Matplotlib by performing array operations, dataset exploration, descriptive statistics, and graphical representation of the data.

Numpy:

➤ **Code:**

```
import numpy
print("ID No: 23IT417")
#creating a simple array
arr=numpy . array ( [1,2,3,4,5])
print(arr)
```

➤ **Output:**

```
ID No: 23IT417
[1 2 3 4 5]
```

➤ **Code:**

```
import numpy as np
print("ID No: 23IT417")
arr=np.array([1,2,3,4,5,6,7,8])

#indexing
print(arr[3])

#backward indexing
print(arr[-2])

#slicing
print(arr[0:4])

#slicing using backward indexing
print(arr[-4 :- 1])

#slicing with skip steps
print(arr[1:6:2])
```

#slicing in 2D array

```
arr1=np.array([[1,2,3,4,5], [6,7,8,9,10]])  
print(arr1[0,1:3])
```

➤ **Output:**

```
ID No: 23IT417  
4  
7  
[1 2 3 4]  
[5 6 7]  
[2 4 6]  
[2 3]
```

➤ **Code:**

```
import numpy as np  
print("ID No: 23IT417")  
arr=np.array(42)  
arr1=np. array([1,2,3])  
arr2=np.array([ [1,2,3], [4,5,6]])  
arr3=np. array ([[1,2,3], [4,5,6] ], [[7,8,9], [10,11,12]]])  
  
print(arr.ndim)  
print(arr1.ndim)  
print(arr2.ndim)  
print(arr3.ndim)
```

➤ **Output:**

```
ID No: 23IT417  
0  
1  
2  
3
```

Pandas**➤ Code:**

```
import pandas as pd
print("ID No: 23IT417")
#series
a=[1,2,3,4,5]
x=pd.Series(a)
print(x)
```

➤ Output:

```
ID No: 23IT417
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

➤ Code:

```
import pandas as pd
print("ID No: 23IT417")
#dataframe
x={
    "name" : ['a', 'b','c'],
    "city" : ['x','y','z']
}
y=pd.DataFrame(x)
print(y)
```

➤ Output:

```
ID No: 23IT417
  name city
0    a    x
1    b    y
2    c    z
```

➤ Code:

```
import numpy as np
print("ID No: 23IT417")
#analysing data
a=[1,4,8,0,5,3,2,6,12]
```

```
print("Array: ",a)
mean=np.mean(a)
print("Mean: ",mean)
median=np.median(a)
print("Median: ",median)
sd=np.std(a)
print("Standard Deviation: ",sd);
```

➤ **Output:**

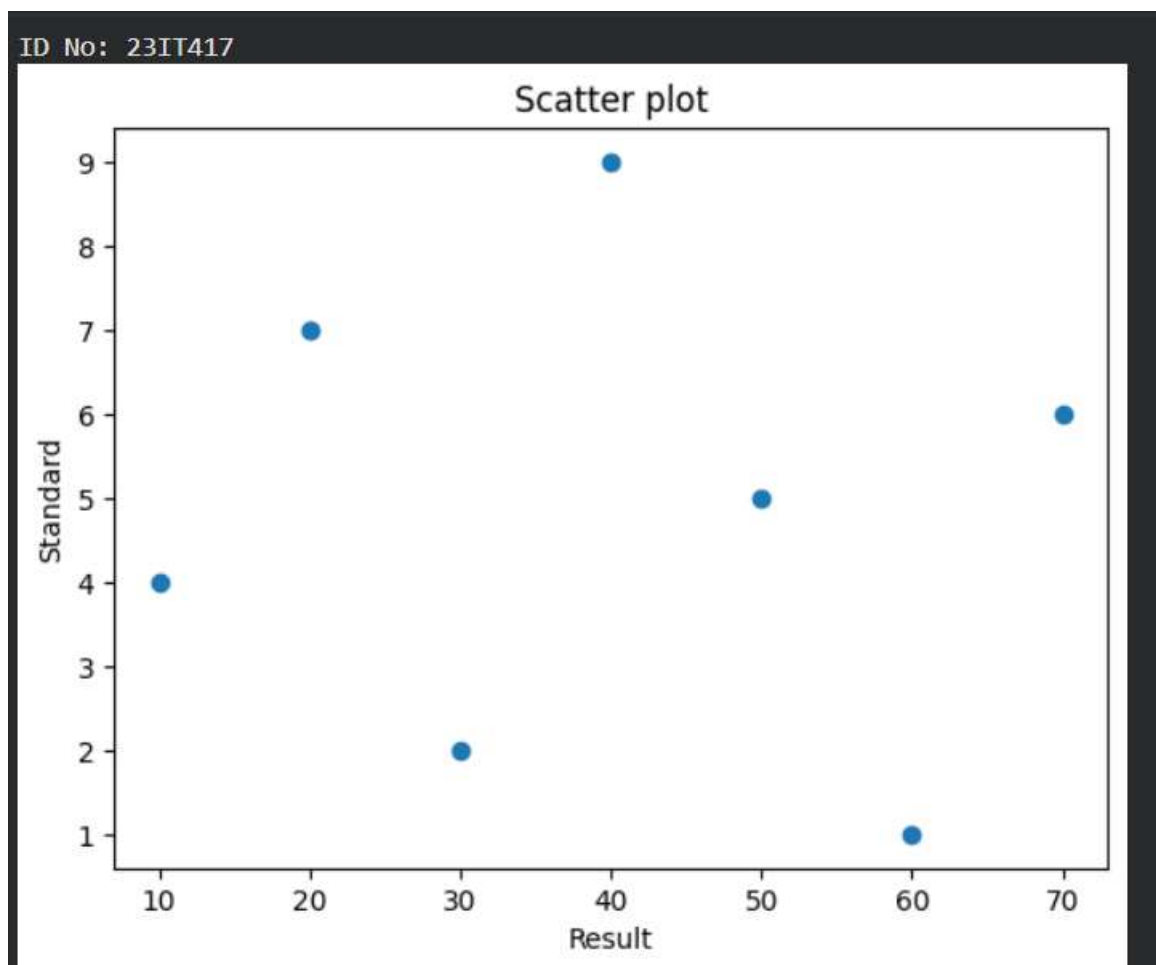
```
ID No: 23IT417
Array: [1, 4, 8, 0, 5, 3, 2, 6, 12]
Mean:  4.555555555555555
Median: 4.0
Standard Deviation:  3.5311663515712675
```

Matplotlib

➤ Code:

```
import matplotlib.pyplot as plt
import numpy as n
print("ID No: 23IT417")
#scatter plot
x=n.array([10,20,30,40,50,60,70])
y=n. array([4,7,2,9,5,1,6])
plt.title("Scatter plot")
plt.xlabel("Result")
plt.ylabel("Standard")
plt.scatter(x,y)
plt.show()
```

➤ Output:



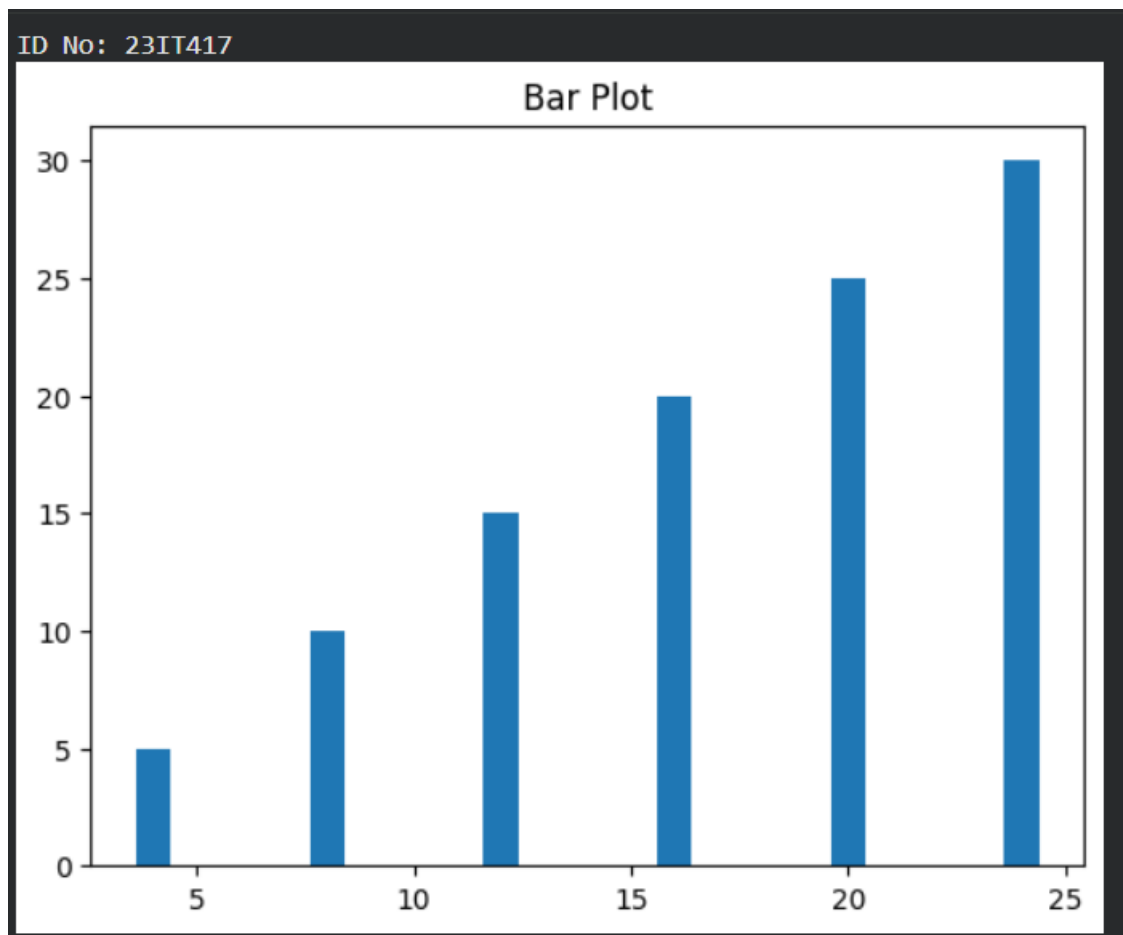
➤ **Code:**

```
#bar plot
import matplotlib.pyplot as plt
import numpy as n
print("ID No: 23IT417")
x=n.array([4,8,12,16,20,24])
y=n.array([5,10,15,20,25,30])

plt.title("Bar Plot")

plt.bar(x,y)
plt.show()
```

➤ **Output:**



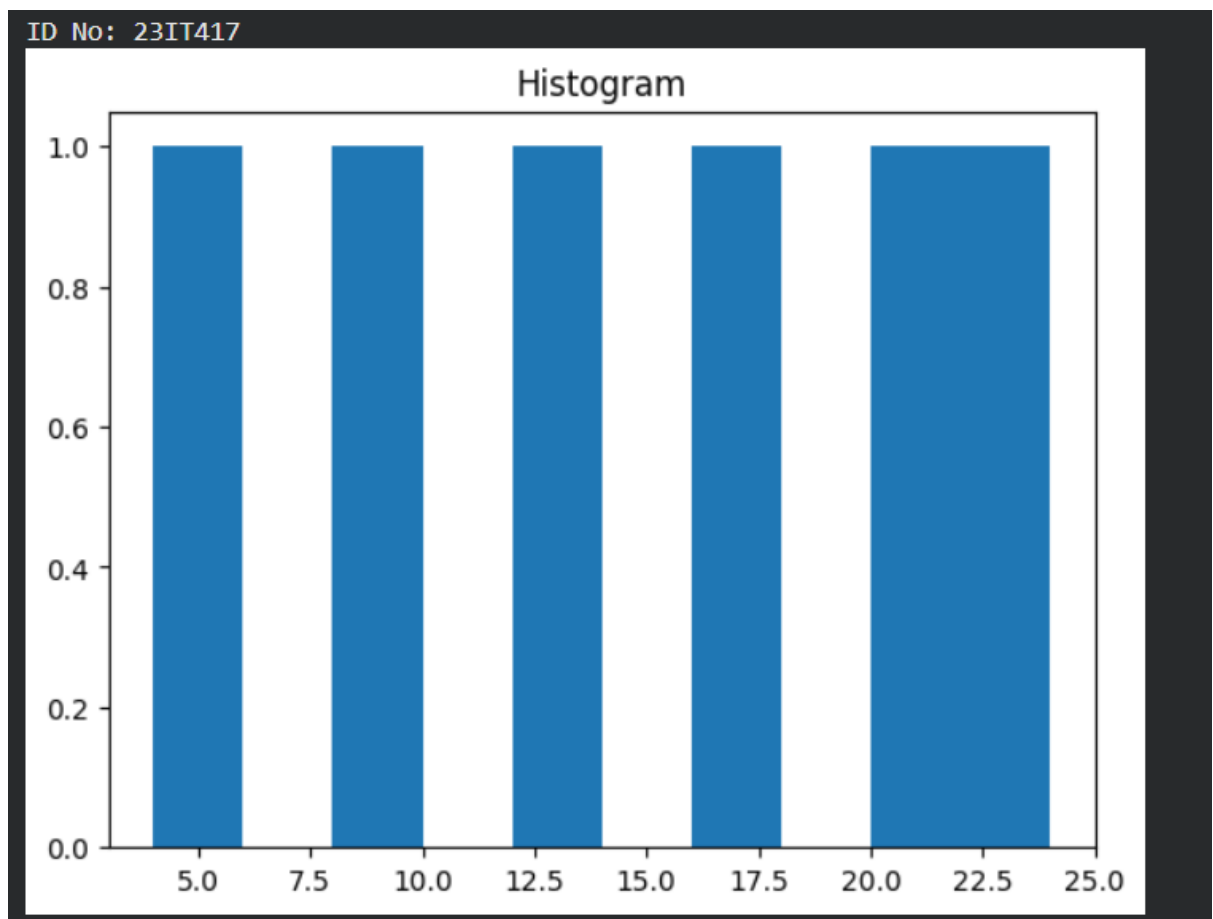
➤ **Code:**

```
#histogram
import matplotlib.pyplot as plt
import numpy as n
print("ID No: 23IT417")
x=n.array([4,8,12,16,20,24])

plt.title("Histogram")

plt.hist(x)
plt.show()
```

➤ **Output:**

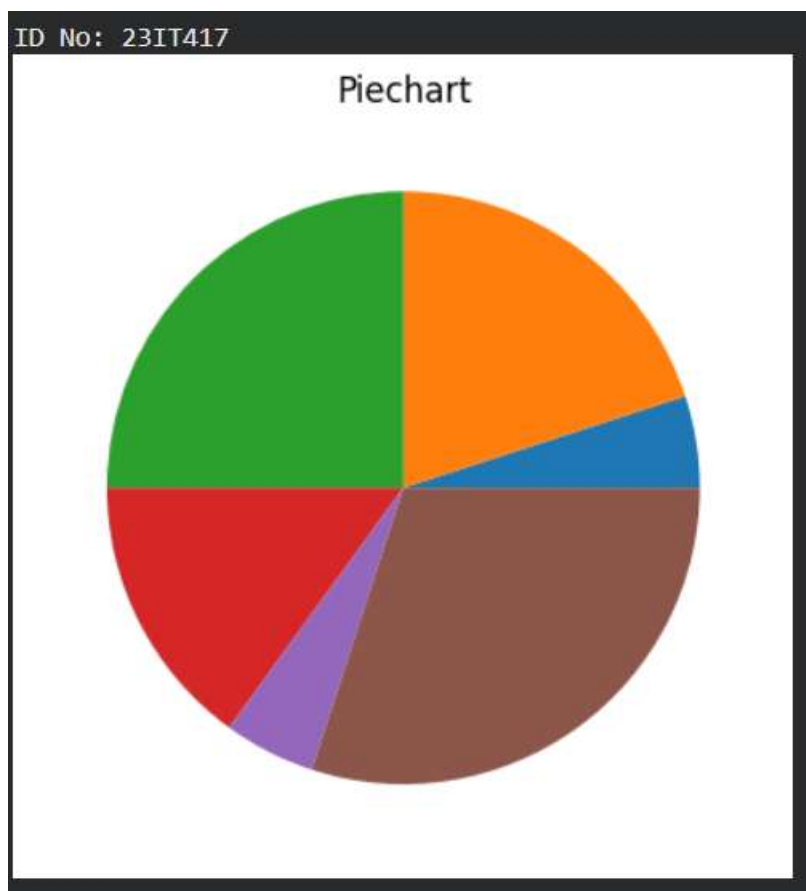


➤ **Code:**

```
#piechart
import matplotlib.pyplot as plt
import numpy as n
print("ID No: 23IT417")
x=n.array([5,20,25,15,5,30])

plt.title("Piechart")
plt.pie(x)
plt.show()
```

➤ **Output:**

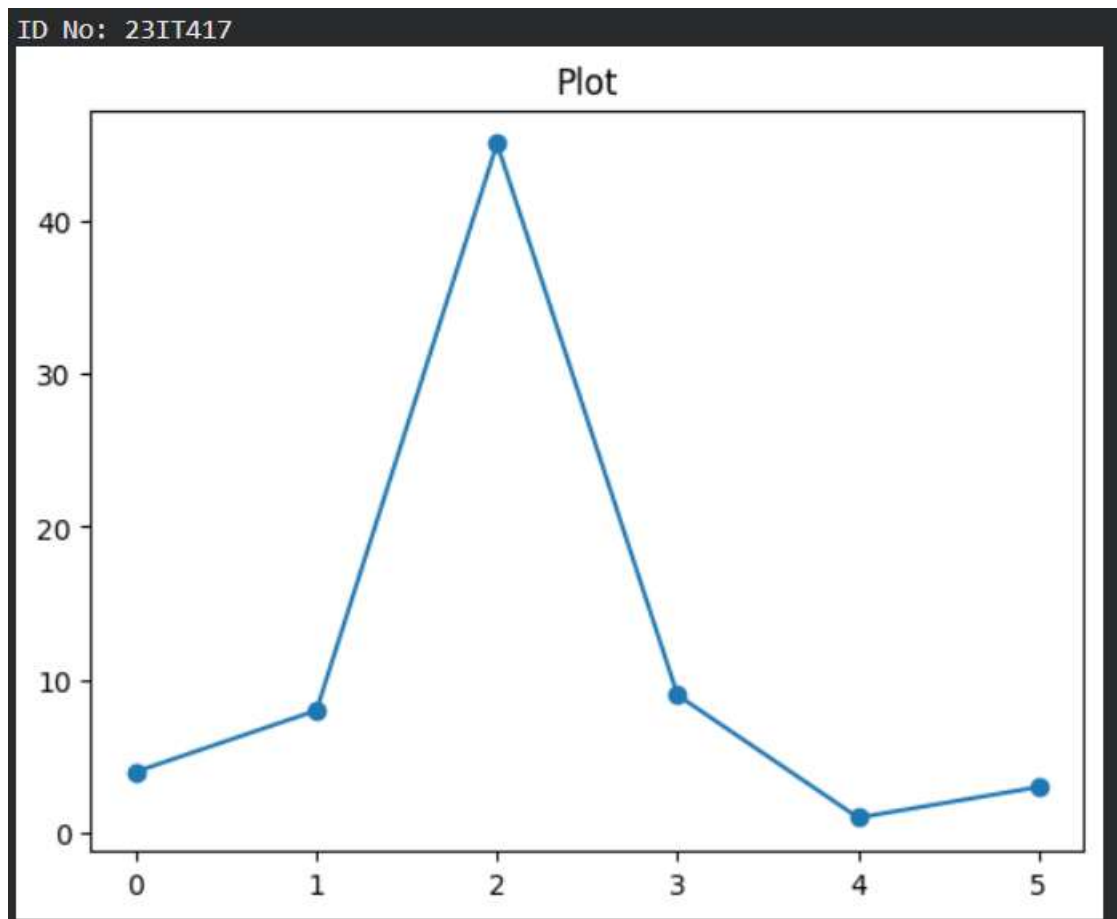


➤ Code:

```
#plot
import matplotlib.pyplot as plt
import numpy as n
print("ID No: 23IT417")
x=n.array([4,8,45,9,1,3])

plt.title("Plot")

plt.plot(x,marker = 'o')
plt.show()
```

➤ Output:

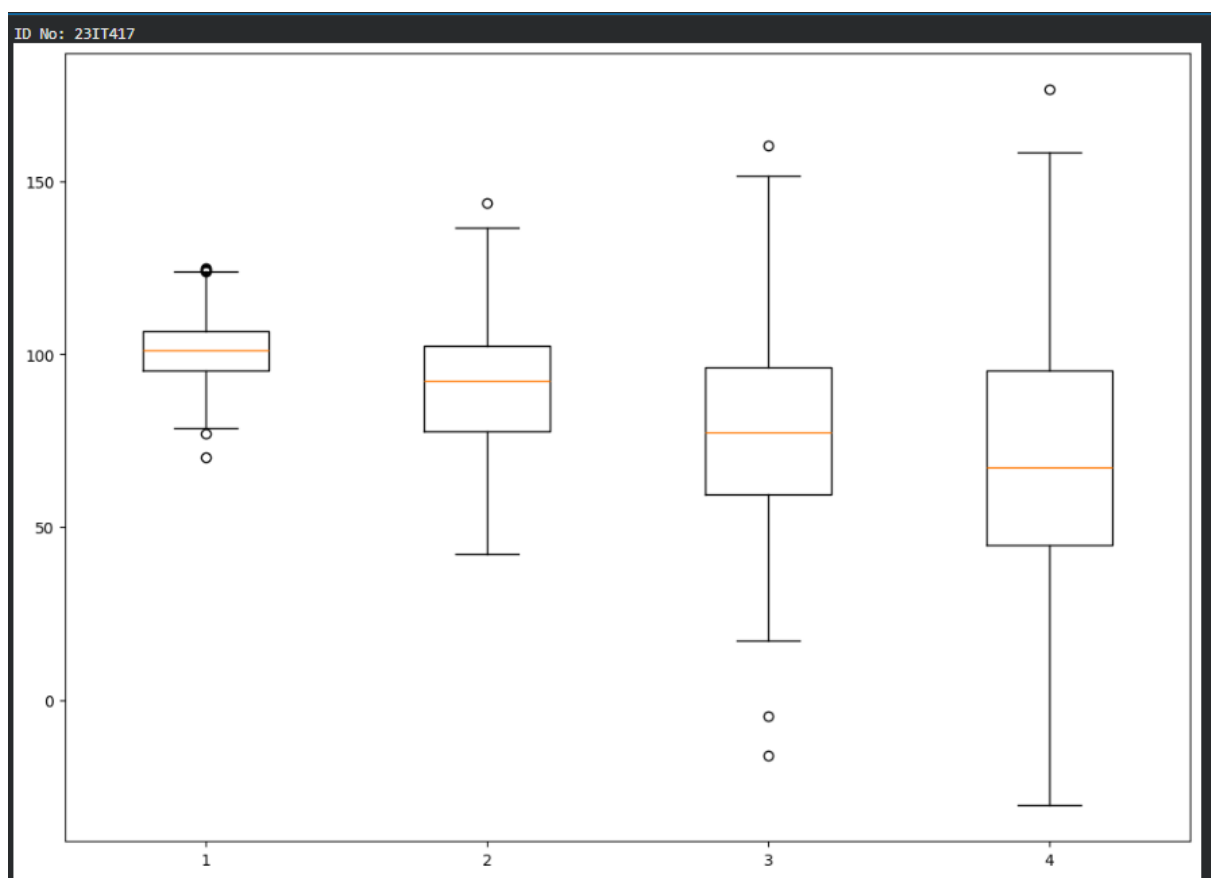
➤ **Code:**

```
#boxplot
import matplotlib.pyplot as plt
import numpy as np
print("ID No: 23IT417")
np.random.seed(10)

d_1 = np.random.normal(100, 10, 200)
d_2 = np.random.normal(90, 20, 200)
d_3 = np.random.normal(80, 30, 200)
d_4 = np.random.normal(70, 40, 208)
d = [d_1, d_2, d_3, d_4]
```

```
fig = plt.figure(figsize=(10, 7))
ax = fig.add_axes([0, 0, 1, 1])
bp = ax.boxplot(d)
```

```
plt.show()
```

➤ **Output:**

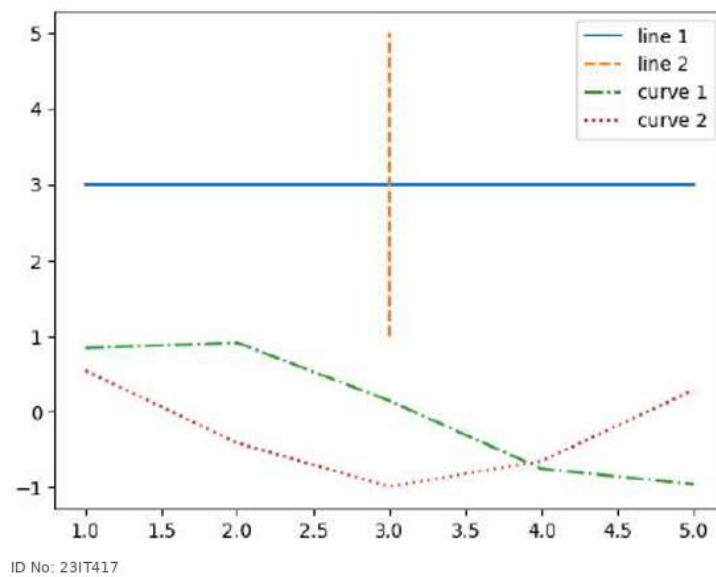
➤ **Code:**

```

import matplotlib.pyplot as plt
import numpy as np
# create data
x = [1,2,3,4,5]
y = [3,3,3,3,3]

# plot Lines
plt.plot(x, y, label = "line 1", linestyle="-")
plt.plot(y, x, label = "line 2", linestyle="-- ")
plt.plot(x, np.sin(x), label = "curve 1", linestyle="-.",)
plt.plot(x, np.cos(x), label = "curve 2", linestyle=":")
plt.legend()
plt.show()
print("ID No: 23IT417")

```

➤ **Output:**

Practical-2

Aim: Perform data preprocessing on a dataset by replacing missing values, removing outliers, applying Min-Max Normalization and Standard using appropriate Python Libraries.

➤ **Code:**

#dataset Loading and performing descriptive statistics

```
import pandas as pd
print("ID No: 23IT417")
df=pd.read_csv("C:/Users/student/23IT417/netflix_dataset.csv")
column_name='rating'
mean_value=df[column_name].mean()
median_value=df[column_name].median()
std_value=df[column_name].std()
mod_value=df[column_name].mode()[0]

print(f"Mean of rating: {mean_value}")
print(f"Median of rating: {median_value}")
print(f"Mode of rating: {mod_value}")
print(f"Standard deviation of rating: {std_value}")
```

➤ **Output:**

```
ID No: 23IT417
Mean of rating: 7.212000000000001
Median of rating: 7.1
Mode of rating: 5.7
Standard deviation of rating: 1.2858420972226332
```

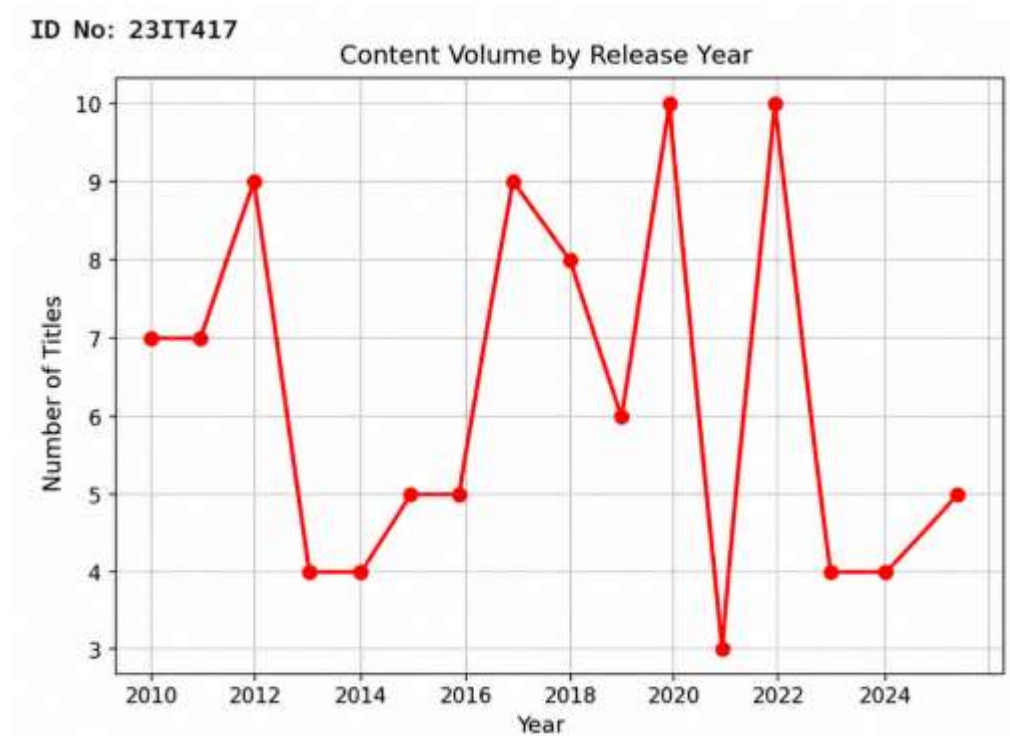
➤ **Code:**

```
import pandas as pd
import matplotlib.pyplot as plt
print("ID No: 23IT417")
# Load dataset
df = pd.read_csv("C:/Users/student/23IT417/netflix_dataset.csv")

# Count shows per year and sort chronologically
df['release_year'].value_counts().sort_index().plot(kind='line', marker='o', color
```

```
# Formatting  
plt.title('Content Volume by Release Year')  
plt.xlabel('Year')  
plt.ylabel('Number of Titles')  
plt.grid(True)  
plt.show()
```

➤ **Output:**



Practical-3

Aim: Implement the Linear Regression algorithm from scratch without using the Scikit-learn library. Then implement the same algorithm using Scikit-learn, compare the predicted values and evaluation metrics (MAE, MSE, RMSE, and R² Score), and visualize the regression line.

➤ **Code:**

Without Scikit-learn library:

```
#with functions
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def load_data(filepath, x_col, y_col):
    #Loads CSV data and extracts X and Y as NumPy arrays.
    df = pd.read_csv(filepath)
    return df[x_col].values, df[y_col].values

def analyze_correlation(X, Y):
    #Calculates and prints the Pearson Correlation Coefficient.
    N = len(X)
    sum_X, sum_Y = np.sum(X), np.sum(Y)
    sum_XY = np.sum(X * Y)
    sum_X_squared = np.sum(X ** 2)
    sum_Y_squared = np.sum(Y ** 2)

    den_r = np.sqrt((N * sum_X_squared - sum_X**2) * (N * sum_Y_squared -
sum_Y**2))

    print("Correlation Analysis")
    if den_r == 0:
        print("Correlation Coefficient (r): Undefined (Zero variance)\n")
        return 0

    r = ((N * sum_XY) - (sum_X * sum_Y)) / den_r
    print(f"Correlation Coefficient (r): {r:.4f}")

    # Interpretation logic
    if abs(r) >= 0.7: strength = "Strong"
    elif abs(r) >= 0.3: strength = "Moderate"
```

```

else: strength = "Weak"

direction = "positive" if r > 0 else "negative" if r < 0 else ""

if r == 0:
    print("Interpretation: No linear relationship.\n")
else:
    print(f"Interpretation: {strength} {direction} linear relationship.\n")
return r

def calculate_regression(X, Y):
    #Calculates slope, intercept, predictions, and evaluation metrics.
    N = len(X)
    sum_X, sum_Y = np.sum(X), np.sum(Y)
    sum_XY = np.sum(X * Y)
    sum_X_squared = np.sum(X ** 2)

    den_m = (N * sum_X_squared - sum_X**2)
    if den_m == 0:
        print("Error: Cannot calculate regression because variance of X is zero.")
        return None

    m = (N * sum_XY - sum_X * sum_Y) / den_m
    c = (sum_Y - m * sum_X) / N

    print("Coefficients")
    print(f"Calculated Slope (m): {m:.4f}")
    print(f"Calculated Intercept (c): {c:.4f}")
    print(f"Regression Equation: Y = {m:.4f} * X + {c:.4f}\n")

    # Performance Evaluation
    Y_pred = m * X + c
    mae = np.mean(np.abs(Y - Y_pred))
    mse = np.mean((Y - Y_pred) ** 2)

    ss_res = np.sum((Y - Y_pred) ** 2)
    ss_tot = np.sum((Y - np.mean(Y)) ** 2)
    r2 = 1.0 if ss_tot == 0 and ss_res == 0 else (0.0 if ss_tot == 0 else 1 - (ss_res / ss_tot))

    print("Performance Evaluation")
    print(f"Mean Absolute Error (MAE): {mae:.4f}")
    print(f"Mean Squared Error (MSE): {mse:.4f}")
    print(f"R-squared Score (R²): {r2:.4f}\n")

```

```

return m, c

def plot_graphs(X, Y, m=None, c=None):
    #Handles both the initial scatter plot and the final regression line plot
    # Plot 1: Initial Scatter Plot
    plt.figure(figsize=(8, 5))
    plt.scatter(X, Y, color='blue', label='Actual data points')
    plt.xlabel('Hours Studied')
    plt.ylabel('Exam Score')
    plt.title('Initial Scatter Plot')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.show()

    # Plot 2: Regression Line (only if m and c are calculated successfully)
    if m is not None and c is not None:
        X_line = np.array([X.min(), X.max()])
        Y_line = m * X_line + c

        plt.figure(figsize=(8, 5))
        plt.scatter(X, Y, color='blue', label='Actual Data')
        plt.plot(X_line, Y_line, color='red', linewidth=2, label=f'Regression Line (Y = {m:.2f}X + {c:.2f})')
        plt.xlabel('Hours Studied')
        plt.ylabel('Exam Score')
        plt.title('Linear Regression Line of Best Fit')
        plt.legend()
        plt.grid(True, linestyle='--', alpha=0.5)
        plt.show()

# Execution block
if __name__ == "__main__":
    # 1. Load data using the function
    X, Y = load_data('Exam.csv', x_col='Hours_Studied', y_col='Exam_Score')

    # 2. Analyze Correlation
    analyze_correlation(X, Y)

    # 3. Calculate Regression Coefficients & Evaluation
    coefficients = calculate_regression(X, Y)

    # 4. Plot results
    if coefficients:
        slope, intercept = coefficients

```

```

    plot_graphs(X, Y, slope, intercept)
else:
    plot_graphs(X, Y)
print("ID No: 23IT417")

```

➤ Output:

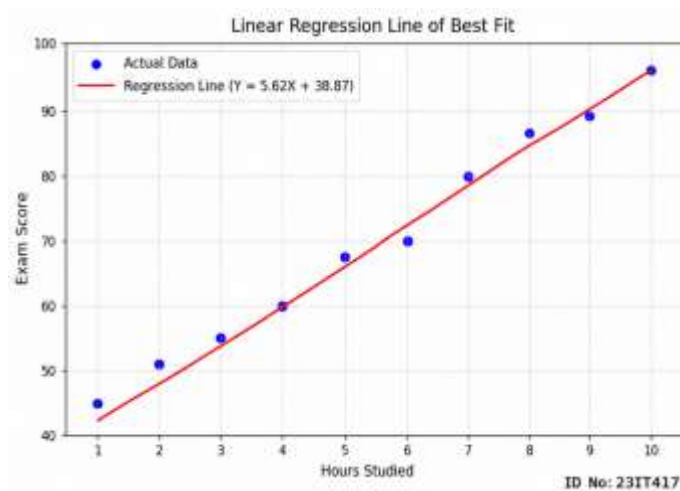
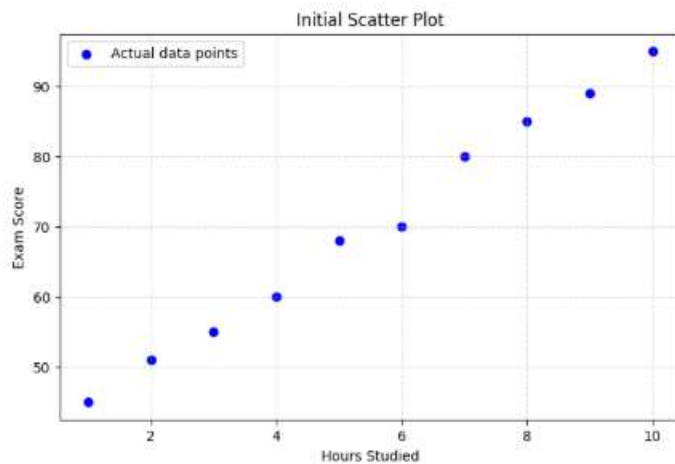
```

*** Correlation Analysis
Correlation Coefficient (r): 0.9970
Interpretation: Strong positive linear relationship.

Coefficients
Calculated Slope (m): 5.6242
Calculated Intercept (c): 38.8667
Regression Equation:  $Y = 5.6242 * X + 38.8667$ 

Performance Evaluation
Mean Absolute Error (MAE): 1.0618
Mean Squared Error (MSE): 1.5952
R-squared Score (R2): 0.9939

```



➤ **Code:****With Scikit-learn library:**

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
print("ID No: 23IT417")

# 1. Load Data & Calculate Correlation
df = pd.read_csv('Exam.csv')
print(f'Correlation: {df.corr().loc['Hours_Studied', 'Exam_Score']:.4f}\n')

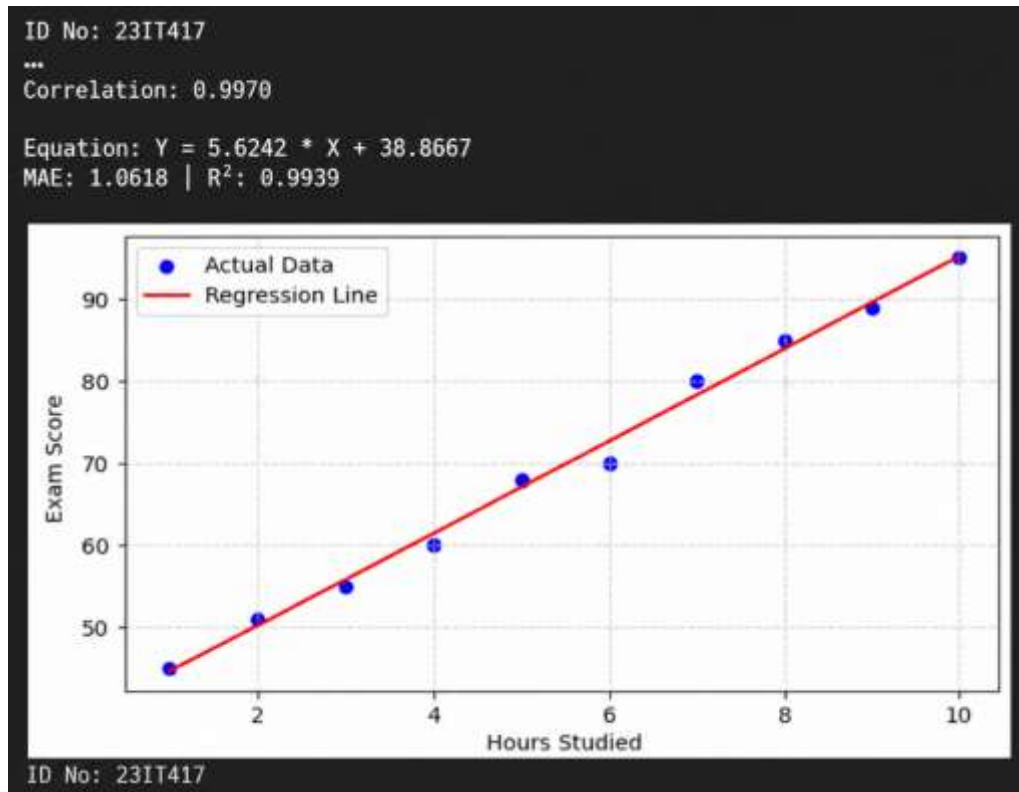
# 2. Fit the Model
X, y = df[['Hours_Studied']], df['Exam_Score'] # X needs to be a DataFrame (2D) for
sklearn
model = LinearRegression().fit(X, y)
y_pred = model.predict(X)

# 3. Print Results
print(f'Equation: Y = {model.coef_[0]:.4f} * X + {model.intercept_:.4f}')
print(f'MAE: {mean_absolute_error(y, y_pred):.4f} | R²: {r2_score(y, y_pred):.4f}\n')

# 4. Plot
plt.figure(figsize=(7, 4))
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, y_pred, color='red', linewidth=2, label='Regression Line')
plt.xlabel('Hours Studied')
plt.ylabel('Exam Score')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
print("ID No: 23IT417")

```

➤ **Output:**



Practical – 4

Aim: Implement the Logistic Regression algorithm using an appropriate dataset. Perform train-test splitting, train the model, predict the class labels for the test data, evaluate the model using appropriate classification metrics, and visualize the confusion matrix.

➤ **Code:**

```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix

# Load Iris dataset
iris = load_iris()

# Features and target
X = iris.data
y = iris.target

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Logistic Regression model
model = LogisticRegression(max_iter=200)

# Train the model
model.fit(X_train, y_train)

# Predict class labels
y_pred = model.predict(X_test)

# Display actual and predicted values
print("Actual Class\tPredicted Class")
print("-----")
```

```

for actual, predicted in zip(y_test, y_pred):
    print(actual, "\t\t", predicted)

# Calculate classification metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Display evaluation metrics
print("\nClassification Metrics")
print("-----")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1 Score  : {f1:.4f}")

# Calculate confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix")
print(cm)

# Plot confusion matrix using normal graph
plt.figure(figsize=(6, 5))
plt.imshow(cm, interpolation='nearest', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.colorbar()

# Add values inside matrix
for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(j, i, cm[i, j], ha='center', va='center')

plt.xticks(range(3), iris.target_names)
plt.yticks(range(3), iris.target_names)
plt.show()
print("ID No: 23IT417")

```

➤ **Output:**

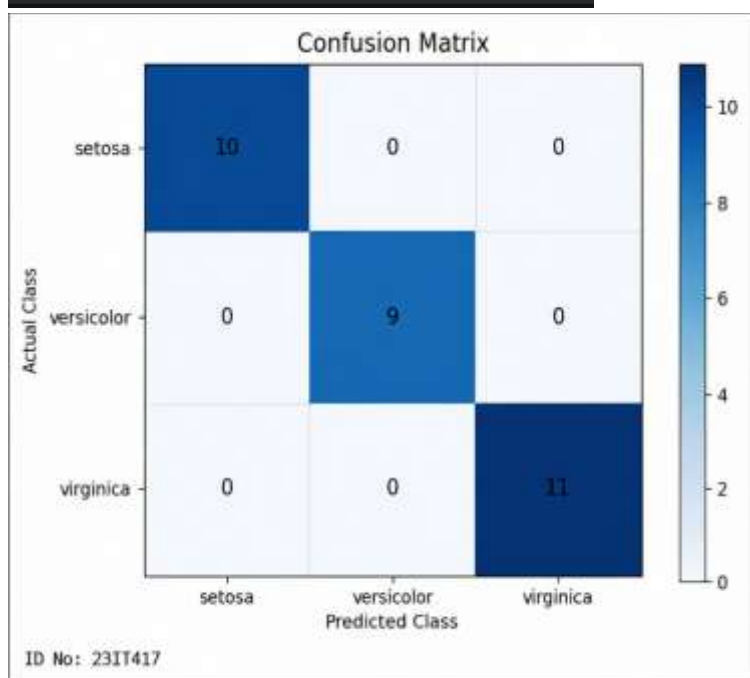
```

Actual Class Predicted Class
-----
...
1 1
0 0
2 2
1 1
1 1
0 0
1 1
2 2
1 1
1 1
2 2
0 0
0 0
0 0
1 1
2 2
1 1
1 1
2 2
0 0
2 2
0 0
2 2
2 2
2 2
2 2
2 2
0 0
0 0

Classification Metrics
-----
Accuracy : 1.0000
Precision : 1.0000
Recall : 1.0000
F1 Score : 1.0000

Confusion Matrix
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

```



Practical – 5

Aim: Write a program to demonstrate working of K-Nearest Neighbors (KNN) classification algorithm from scratch and using the Scikit-learn library on an appropriate dataset. Compare both implementations based on predicted class labels and classification performance metrics.

➤ **Code:**

From scratch:

```
from ucimlrepo import fetch_ucirepo
import pandas as pd
import math
print("ID No:23IT417")
# Load Iris Dataset
iris = fetch_ucirepo(id=53)

# Features and Target
X = iris.data.features
y = iris.data.targets

# Combine features and target
data = pd.concat([X, y], axis=1)

# Split dataset
train = data.iloc[:120]
test = data.iloc[120:]

# Input K value
k = int(input("Enter K Value: "))

correct = 0

print("\nActual Class\t\tPredicted Class")
print("-----")

# Test each record
for i in range(len(test)):

    distances = []

    # Calculate Euclidean distance
```

```

for j in range(len(train)):

    d = math.sqrt(
        (test.iloc[i, 0] - train.iloc[j, 0]) ** 2 +
        (test.iloc[i, 1] - train.iloc[j, 1]) ** 2 +
        (test.iloc[i, 2] - train.iloc[j, 2]) ** 2 +
        (test.iloc[i, 3] - train.iloc[j, 3]) ** 2
    )

    distances.append([d, train.iloc[j, 4]])

# Sort according to distance
distances.sort()

# Count classes of K nearest neighbours
count = {}

for x in range(k):
    label = distances[x][1]

    if label in count:
        count[label] += 1
    else:
        count[label] = 1

# Predict class with maximum votes
prediction = max(count, key=count.get)

actual = test.iloc[i, 4]

print(actual, "\t", prediction)

if actual == prediction:
    correct += 1

# Calculate Accuracy
accuracy = (correct / len(test)) * 100

print("\n-----")
print("Total Test Records    :", len(test))
print("Correct Predictions    :", correct)
print("Incorrect Predictions   :", len(test) - correct)
print("Accuracy                : {:.2f}%".format(accuracy))

```

➤ **Output:**

```

ID No: 23IT417
Enter K Value: 3
..
Actual Class      Predicted Class
-----
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-versicolor
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-versicolor
Iris-virginica    Iris-versicolor
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-versicolor
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-versicolor
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-versicolor
Iris-virginica    Iris-virginica
Iris-virginica    Iris-virginica
Iris-virginica    Iris-versicolor

-----
Total Test Records      : 30
Correct Predictions     : 23
Incorrect Predictions   : 7
Accuracy                : 76.67%

```

➤ **Code:**
With Scikit-learn library

```

from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
print("ID No:23IT417")
# Load Iris Dataset
iris = fetch_ucirepo(id=53)

# Features and Target
X = iris.data.features
y = iris.data.targets

# Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Input K value
k = int(input("Enter K Value: "))

# Create KNN model
knn = KNeighborsClassifier(n_neighbors=k)

# Train model
knn.fit(X_train, y_train.values.ravel())

# Predict
y_pred = knn.predict(X_test)

# Display results
print("\nActual Class\t\tPredicted Class")
print("-----")

for actual, predicted in zip(y_test.values.ravel(), y_pred):
    print(f'{actual:<20} {predicted}')

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\n-----")

```

```
print("Total Test Records :", len(y_test))
print("Correct Predictions:", sum(y_test.values.ravel() == y_pred))
print("Accuracy          : {:.2f}%".format(accuracy * 100))
```

➤ **Output:**

```
ID No: 23IT417
Enter K Value: 3
...
Actual Class      Predicted Class
-----
Iris-versicolor   Iris-versicolor
Iris-setosa        Iris-setosa
Iris-virginica     Iris-virginica
Iris-versicolor   Iris-versicolor
Iris-versicolor   Iris-versicolor
Iris-setosa        Iris-setosa
Iris-versicolor   Iris-versicolor
Iris-virginica     Iris-virginica
Iris-versicolor   Iris-versicolor
Iris-versicolor   Iris-versicolor
Iris-virginica     Iris-virginica
Iris-setosa        Iris-setosa
Iris-setosa        Iris-setosa
Iris-setosa        Iris-setosa
Iris-setosa        Iris-setosa
Iris-versicolor   Iris-versicolor
Iris-virginica     Iris-virginica
Iris-versicolor   Iris-versicolor
Iris-versicolor   Iris-versicolor
Iris-virginica     Iris-virginica
Iris-setosa        Iris-setosa
Iris-virginica     Iris-virginica
Iris-setosa        Iris-setosa
Iris-virginica     Iris-virginica
Iris-virginica     Iris-virginica
Iris-virginica     Iris-virginica
Iris-virginica     Iris-virginica
Iris-virginica     Iris-virginica
Iris-setosa        Iris-setosa
Iris-setosa        Iris-setosa
-----
Total Test Records : 30
Correct Predictions : 30
Accuracy           : 100.00%
```

Practical – 6

Aim: Write a program to demonstrate working of K-Nearest Neighbors (KNN) classification algorithm from scratch and using the Scikit-learn library on an appropriate dataset. Compare both implementations based on predicted class labels and classification performance metrics.

➤ **Code:**

```
# DECISION TREE CLASSIFICATION USING KAGGLE IRIS DATASET
```

```
# 1. Install KaggleHub
```

```
!pip install -q kagglehub
```

```
# 2. Import libraries
```

```
import kagglehub
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.tree import plot_tree
```

```
from sklearn.metrics import (
```

```
    accuracy_score,
```

```
    confusion_matrix,
```

```
    classification_report
```

```
)
```

```
print("ID NO:23IT417")
```

```
# 3. Download Iris dataset from Kaggle
```

```
path = kagglehub.dataset_download("uciml/iris")
```

```
print("Dataset downloaded successfully!")
```

```
print("Dataset path:", path)
```

```
# 4. Load dataset
```

```
df = pd.read_csv(path + "/Iris.csv")
```

```
print("\nFirst 5 rows:")
```

```
print(df.head())
```

```

# 5. Dataset information
print("\nDataset Shape:")
print(df.shape)

print("\nColumns:")
print(df.columns)

print("\nMissing Values:")
print(df.isnull().sum())

# 6. Select features
X = df[
    [
        "SepalLengthCm",
        "SepalWidthCm",
        "PetalLengthCm",
        "PetalWidthCm"
    ]
]
y = df["Species"]

# 7. Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining data:", X_train.shape)
print("Testing data:", X_test.shape)

# MODEL 1: ENTROPY / INFORMATION GAIN
dt_entropy = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

# Train model
dt_entropy.fit(X_train, y_train)

```

```

# Predict test data
y_pred_entropy = dt_entropy.predict(X_test)

# Accuracy
accuracy_entropy = accuracy_score(
    y_test,
    y_pred_entropy
)

print("\n=====")
print("DECISION TREE USING ENTROPY")
print("=====")

print("Accuracy:",
      accuracy_entropy)

print("Accuracy Percentage:",
      accuracy_entropy * 100, "%")

# Confusion Matrix
print("\nConfusion Matrix:")
print(
    confusion_matrix(
        y_test,
        y_pred_entropy
    )
)

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred_entropy
    )
)

# MODEL 2: GINI INDEX

dt_gini = DecisionTreeClassifier(
    criterion="gini",
    random_state=42

```

```

)

# Train model
dt_gini.fit(X_train, y_train)

# Predict test data
y_pred_gini = dt_gini.predict(X_test)

# Accuracy
accuracy_gini = accuracy_score(
    y_test,
    y_pred_gini
)

print("\n=====")
print("DECISION TREE USING GINI INDEX")
print("=====")

print("Accuracy:",
      accuracy_gini)

print("Accuracy Percentage:",
      accuracy_gini * 100, "%")

# Confusion Matrix
print("\nConfusion Matrix:")
print(
    confusion_matrix(
        y_test,
        y_pred_gini
    )
)

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred_gini
    )
)

```

```

print("\n=====")
print("MODEL COMPARISON")
print("=====")

print(
    "Entropy / Information Gain Accuracy:",
    accuracy_entropy * 100,
    "%"
)

print(
    "Gini Index Accuracy:",
    accuracy_gini * 100,
    "%"
)
# VISUALIZE ENTROPY TREE
plt.figure(figsize=(20, 10))

plot_tree(
    dt_entropy,
    feature_names=X.columns,
    class_names=dt_entropy.classes_,
    filled=True,
    rounded=True
)

plt.title(
    "Decision Tree using Entropy (Information Gain)"
)

plt.show()

# VISUALIZE GINI TREE

plt.figure(figsize=(20, 10))

plot_tree(
    dt_gini,
    feature_names=X.columns,
    class_names=dt_gini.classes_,
    filled=True,
    rounded=True
)

```

```
plt.title(
    "Decision Tree using Gini Index"
)

plt.show()
```

➤ Output:

```
ID NO:23IT417
Using Colab cache for faster access to the 'iris' dataset.
Dataset downloaded successfully!
Dataset path: /kaggle/input/iris

First 5 rows:
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Species
0   1             5.1           3.5           1.4           0.2  Iris-setosa
1   2             4.9           3.0           1.4           0.2  Iris-setosa
2   3             4.7           3.2           1.3           0.2  Iris-setosa
3   4             4.6           3.1           1.5           0.2  Iris-setosa
4   5             5.0           3.6           1.4           0.2  Iris-setosa

Dataset Shape:
(150, 6)

Columns:
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')

Missing Values:
Id           0
SepalLengthCm  0
SepalWidthCm  0
PetalLengthCm  0
PetalWidthCm  0
Species       0
dtype: int64
```

```

Training data: (120, 4)
Testing data: (30, 4)

=====
DECISION TREE USING ENTROPY
=====
Accuracy: 0.9333333333333333
Accuracy Percentage: 93.33333333333333 %

```

Confusion Matrix:

```

[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

```

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	0.90	0.90	0.90	10
Iris-virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

```

=====
DECISION TREE USING GINI INDEX
=====
Accuracy: 0.9333333333333333
Accuracy Percentage: 93.33333333333333 %

```

Confusion Matrix:

```

[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

```

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	0.90	0.90	0.90	10
Iris-virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

```

=====
MODEL COMPARISON
=====

```

```

Entropy / Information Gain Accuracy: 93.33333333333333 %
Gini Index Accuracy: 93.33333333333333 %

```

